Build Instrument: Build Spec Template

Chapter: 07 — Build **Artifact produced:** Completed Build Spec (8 sections) + Guardrails Checklist + Implementation Mistakes audit **Who's in the room:** Constraint owner from Signal + whoever will build or configure the solution (internal team, vendor, or low-code assembler) + the human supervisor named in the Hybrid Accountability Chart from Design

Purpose

Build translates a locked design into a working, deployed system. The Build Spec is the artifact that makes that translation unambiguous. Every section answers a question the builder will otherwise have to invent an answer to mid-build — and every invented answer is a design decision made under pressure by the wrong person at the wrong time. You leave this worksheet with a spec clear enough that the builder executes against it without negotiating requirements.

Part 1: Choose Your Build Path

Before you write the spec, determine which build path fits the design. The design determines the path — not a vendor relationship, an internal preference, or a market trend.

Run through these three questions in order. Stop at the first "yes."

Decision tree:

1. **Does a mature product already do what the designed workflow requires, with light configuration?** If yes: **Off-the-shelf.** Buy, configure, connect. The work is selection and integration, not construction.
2. **Is the workflow custom but composed of standard moves — pull from a system, run through an agent, write back, notify someone?** If yes: **Low-code.** Zapier, Make, n8n, Airtable + AI layer, Claude Projects with skills. Your team owns it directly and can change it without an engineer.
3. **Does the workflow run at a scale, or touch a system of record, that low-code tools cannot reach? Does data sensitivity rule out a third-party automation platform?** If yes: **Hand-built.** Custom development with AI APIs, usually with an engineer in the loop.

Most sprints land in low-code. Hand-built is less common than people assume. Off-the-shelf is less common than people hope.

> **Pro Tip:** The build path is chosen AFTER Design, not before. Most companies pick their tools first and then try to design a workflow around the tool's limitations. That is backwards. Design names the shape. Build finds the path that fits it.

> **Action Step:** Pull the designed workflow from Design. Run through the three questions above. Write down which path you're on and why. If you can't answer the questions without guessing, Design isn't done — go back.

Part 2: Write the Build Spec

Eight sections. Every section must be completed before building starts. An empty section is a decision someone will make under pressure later — and they will make it wrong.

Section 1: Workflow Summary

What does this workflow do, start to finish? Three to five sentences. Plain language, no jargon. A builder who knows nothing about your business should be able to read this and understand what they are constructing.

If you cannot explain it in five sentences, the design is not clear enough. Go back to Design.

Section 2: Inputs

What data, documents, or triggers start the workflow? Where do they come from? In what format? How does the builder access them?

List every input. Be specific — not "customer data" but "inbound brief submitted via web form, stored in CRM as a new deal record, containing project description, timeline, and budget range."

Section 3: Outputs

What does the workflow produce? In what form? Where does it go? Who consumes it?

The output must land in a form the human supervisor can evaluate the same way they would evaluate a person's work. If the output requires the supervisor to learn a new tool or check a new system, the output is in the wrong place.

Section 4: Agent Scope

What does the AI agent do in this workflow? What decisions does it make? What decisions does it NOT make?

Be explicit about the boundary. "The agent drafts the quote" is not a scope statement. "The agent drafts the quote using the rate card and historical quote library; it does not set final pricing, does not override the rate card, and does not commit the quote to the client" is a scope statement.

Section 5: Human Supervisor Role

Who reviews agent output? How often? What are they reviewing for? What triggers escalation?

Reference the Hybrid Accountability Chart from Design. The supervisor named there is the person named here. If the names don't match, reconcile before you build.

Section 6: Systems and Integrations

What tools and platforms are involved? What connects to what? Read access, write access, or both? Authentication method? Data sensitivity classification for each system?

Reference your Knowledge Map pipeline status from Source. Every system marked “Connected” is an integration the build can lean on. Every system marked “Manual” or “Broken” is an integration the build must address or work around.

Section 7: Failure Modes

What happens when the agent gets it wrong? What happens when data is missing? What happens when a system is down?

For each failure mode, specify: who gets notified, in what form, on what timeline, and what the fallback is. If you cannot describe what happens when it breaks, you cannot ship it.

Section 8: Build Path and Constraints

Which of the three paths from Part 1? Why? What constraints must the builder observe — budget, timeline, data residency, existing licenses, internal security requirements, team technical capability?

> **Action Step:** Open a document. Write the eight section headers. Fill in Sections 1-3 from your Design artifacts. If you cannot fill in Sections 1-3 without guessing, Design is not done — go back. Then complete Sections 4-8 using the Hybrid Accountability Chart and Knowledge Map. Every section must have content before the spec goes to a builder.

Part 3: The Implementation Mistakes Checklist

Before building starts, audit the spec against these seven common failures. Each one has ended a sprint early or produced a system that got shut down within weeks.

- [ ] **Reinventing existing capabilities.** Search before you build. Does a tool, feature, or integration already exist that does what this section of the spec describes? If yes, use it. Building what already exists is the most expensive form of wasted time.
- [ ] **Using the wrong tool for the job.** Match the tool to the build path, not the other way around. If the spec calls for low-code and someone is proposing a custom API integration because “it would be cleaner,” the spec wins. If the spec calls for hand-built and someone is trying to force it into Zapier because the team already has a license, the spec wins.
- [ ] **Building before the spec is complete.** Every empty section in the spec is a decision the builder will make on the fly. Builders are not designers. The decisions they make under build pressure will not match what Design intended. Complete the spec first.
- [ ] **Ignoring the handoff.** The workflow must connect to the systems people already use. If the output lands in a new tool the team has never opened, adoption is zero. The output goes where the work already lives.

- [ ] **Skipping failure modes.** If Section 7 is empty or vague, the build will handle the happy path and break on the first edge case. Every workflow has edge cases. Name them before they name themselves.
- [ ] **Building for the demo instead of the workflow.** A demo shows the happy path. A build handles the edge cases, the missing data, the system outages, the weird inputs, the supervisor who's on vacation. If the build only works on the example you prepared, it is a demo.
- [ ] **Declaring done before testing against real inputs.** Pull last week's actual data — the real records, the real emails, the real requests — and run them through the build. If you tested against synthetic data or cherry-picked examples, you have not tested.

> **Pro Tip:** The most common implementation failure is building something that already exists because nobody searched first. Before every build, spend fifteen minutes asking: does this already exist as a feature in a tool we own?

> **Action Step:** Walk through all seven items with the builder present. Check each one against the completed spec. Any item that cannot be checked off is a gap that must be resolved before build begins.

Part 4: The Guardrails Checklist

Seven questions. All must have documented answers before the solution goes live. These are the conditions under which the human supervisor can actually supervise.

1. **What data does the agent access?** Explicit list. Not "whatever it needs" — name every data source, every system, every field. What is it permitted to read? What is it permitted to write? What is it explicitly not permitted to touch?
2. **What can the agent do without human approval?** Explicit boundaries. The agent can draft a quote. The agent cannot send a quote. The agent can pull pricing from the rate card. The agent cannot override the rate card. Write it at that level of specificity.
3. **What requires human sign-off before the agent acts?** Explicit list. Every action that changes a record of consequence, sends a communication to a client, commits a number, or creates a deliverable — does it require human review first?
4. **What happens when the agent encounters an input it was not designed for?** The agent will receive inputs outside its training. What does it do? Refuse and escalate? Attempt and flag? Proceed silently? The answer must be designed, not discovered.
5. **How is agent output quality measured?** Not "we'll review it." What specific checks? Accuracy against what baseline? Completeness measured how? Consistency verified against what standard? If the measurement is "someone looks at it," define what they are looking for.
6. **Who is responsible when the agent produces a bad output?** A named person — one human who owns the output the same way they would own it if a direct report produced it. This is the human supervisor from the Hybrid Accountability Chart.

7. **What would cause you to shut down the agent workflow immediately?** Kill switch criteria. Define the conditions in advance — not in the moment when something has already gone wrong. What error rate? What type of error? What data exposure? What customer impact? Write the criteria now, when you are thinking clearly.

> **Pro Tip:** Guardrails are what make the workflow trustable. An agent without them gets shut down the first time something goes wrong — and then the constraint goes back to costing what it cost before.

> **Action Step:** Answer all seven questions in writing. Attach the answers to the Build Spec as a ninth section or a companion document. If any question cannot be answered, the build is not ready to deploy — resolve the gap before going live.

Part 5: The "Done" Test

Build is done when the system handles real work, not when it handles the demo.

Pull the last week of actual inputs the workflow was supposed to handle — the real records, the real requests, the real data. Run them through the build. Answer these four questions:

1. **Did it produce the expected outputs?** Compare agent output to what a competent person would have produced. Not identical — but within the quality range the supervisor would accept from a team member.
2. **Did it handle the edge cases in your Failure Modes section?** Every failure mode you named in Section 7 of the spec — did the build handle it as specified? If you named five failure modes and tested against three, you are not done.
3. **Did it escalate correctly when it should have?** Feed it inputs that should trigger escalation. Did the right person get notified, in the right form, on the right timeline?
4. **Could the human supervisor understand and act on its output?** Show the output to the named supervisor. Without explanation. Without a walkthrough. Can they evaluate it and make a decision? If they need you to explain what the output means, the output is not ready.

If yes to all four: Build is done. Hand off to Deliver.

If no to any: fix what failed. Run the test again. Do not advance past Build on a partial pass.

> **Action Step:** Pull last week's real inputs. Run the full test. Document the results for each of the four questions. Fix any failures and retest. The test results are part of the Build handoff to Deliver.

Populated Example: Manufacturing Quoting — Rapid Estimate Workflow

Constraint (from Signal): Design engineers spend 4-6 hours producing initial designs for every bid, including the 50%+ that won't convert, at an estimated cost of $180K/year in misallocated engineering time.

Build path chosen: Low-code. Off-the-shelf CRM already in place (not replaced). AI agent layer added via low-code platform connecting CRM to quoting workflow. No custom engineering required.

Section 1: Workflow Summary

When a new bid request arrives in the CRM, the quoting agent drafts a rapid cost estimate using the historical quote library, rate card, and customer segment data. The estimate lands in the project lead's review queue within 15 minutes. The project lead reviews, edits if needed, and routes: high-confidence estimates go to the principal for final sign-off; low-confidence estimates get flagged for full engineering review. The principal sees only final-stage quotes, not first drafts.

Section 2: Inputs

- **Inbound bid brief:** Submitted via web form or email, parsed into CRM as a new deal record. Contains project description, timeline, approximate scope, and customer name.
- **Historical quote library:** 3 years of past quotes stored in shared drive (PDF and spreadsheet). Indexed by project type, customer segment, and final price.
- **Rate card:** Current pricing maintained by finance in ERP. Updated quarterly.
- **Customer segment data:** CRM field — existing customer, new customer, strategic account. Determines margin rules.
- **Win/loss history:** CRM records showing conversion rates by segment, project type, and quote turnaround time.

Section 3: Outputs

- **Rapid cost estimate:** Lands in project lead's CRM review queue as a draft record. Contains: estimated price range (not a single number), line-item breakdown, confidence flag (high/medium/low), and the three most similar historical quotes used as basis.
- **Routing recommendation:** High-confidence estimates recommended for principal sign-off. Low-confidence estimates recommended for full engineering review. Routing is a recommendation — the project lead makes the final call.

Section 4: Agent Scope

The agent: - Matches the inbound brief to the three most similar historical quotes by project type and scope - Applies current rate card pricing to produce an estimated range - Flags confidence level based on similarity to historical quotes (high = 90%+ match to past project; medium = 70-89%; low = below 70%) - Drafts the estimate with line-item breakdown

The agent does NOT: - Set a final price (the estimate is a range, not a commitment) - Override the rate card for any reason - Apply customer-specific pricing exceptions (those require the ops manager) - Send anything to the client - Access financial data beyond the rate card and historical quotes

Section 5: Human Supervisor Role

Project lead reviews every estimate before it leaves the queue. Reviews for: accurate scope interpretation, reasonable pricing range, correct confidence flag, appropriate routing recommendation. Expected review time: 10-15 minutes per estimate (vs. 4-6 hours for full engineering estimate).

Escalation triggers: - Confidence flag is "low" — project lead routes to engineering for full estimate - Customer has pricing exceptions on file — project lead routes to ops manager - Estimate exceeds $50K — project lead routes to principal regardless of confidence

Principal signs off on final quotes only. Does not review drafts.

Section 6: Systems and Integrations

SYSTEM	ACCESS	PIPELINE STATUS (FROM SOURCE)	SENSITIVITY
CRM (deal records, win/loss)	Read + Write (draft estimates)	Manual — needs API connection built	Low — no PII beyond contact names
ERP (rate card)	Read only	Connected to invoicing; new read connection needed for quoting	Medium — pricing data
Shared drive (historical quotes)	Read only	Broken — needs indexing and connection	Low — internal project data
Email (inbound briefs)	Read only (parsed into CRM)	Connected via existing CRM integration	Low

Section 7: Failure Modes

FAILURE	WHAT HAPPENS	WHO'S NOTIFIED	FALLBACK
Agent cannot find similar historical quotes (confidence below 50%)	Estimate not generated; bid routed directly to engineering	Project lead via CRM notification	Full engineering estimate (existing process)
Rate card data is stale or missing for a line item	Agent flags the line item as "pricing unavailable" and does not estimate it	Project lead + finance via email	Finance provides current pricing; project lead completes estimate manually
CRM integration is down	Inbound briefs queue in email; no auto-parsing	Ops manager via system alert	Manual entry into CRM; agent processes once connection restores
Agent produces an estimate that is off by more than 25% from the historical baseline	Confidence flag should catch this (low confidence), but if it doesn't, project lead's review is the safety net	Project lead catches in review	Project lead rejects and routes to engineering

Section 8: Build Path and Constraints

Path: Low-code. CRM + AI agent layer via Make (automation platform the team already licenses).

Why not off-the-shelf: No quoting product maps to this firm's specific workflow — the combination of historical quote matching, rate card application, and confidence-based routing is custom to this operation.

Why not hand-built: The workflow is composed of standard moves (read from CRM, read from file store, run through agent, write back to CRM, notify). No scale or data sensitivity issues that require custom code.

Constraints: - Budget: Under $500/month in tooling costs (Make plan + AI API usage) - Timeline: Three weeks from spec completion to deployment - Data residency: All data stays within existing systems; no new data stores created - The historical quote library (shared drive) must be indexed before build begins — this is a prerequisite, not a build task

---

Guardrails for the Quoting Agent

1. **Data access:** CRM deal records (read/write), ERP rate card (read), historical quote library on shared drive (read). No access to: financial reporting, employee data, client contracts, or any system not listed.
2. **Without human approval:** Match historical quotes, apply rate card, generate estimate draft, assign confidence flag, place draft in review queue.
3. **Requires human sign-off:** Sending any estimate to a client. Applying pricing exceptions. Routing decisions (project lead confirms or overrides the agent's routing recommendation). Any quote over $50K.
4. **Unrecognized input:** If the inbound brief does not contain enough information to match against historical quotes, the agent does not guess. It flags the brief as "insufficient for rapid estimate" and routes it to the project lead for manual handling.
5. **Quality measurement:** Accuracy is measured weekly by comparing rapid estimates to final quoted prices on deals that converted. Target: rapid estimates within 15% of final price on high-confidence quotes. Tracked in a simple spreadsheet by the ops manager.
6. **Responsible person:** Project lead owns every estimate the agent produces. The agent's output is the project lead's output. If a bad estimate goes to a client, the project lead is accountable — same as if a junior team member had drafted it.
7. **Kill switch:** Agent is shut down immediately if: (a) three estimates in one week are off by more than 30% from final price, (b) any estimate is sent to a client without human review, or (c) the agent accesses data outside its defined scope. Ops manager holds the kill switch.

Summary of Action Steps

1. **Choose your build path** — run the three-question decision tree against the designed workflow (Part 1)
2. **Write the Build Spec** — eight sections, all completed, no blanks (Part 2)
3. **Audit against implementation mistakes** — seven-item checklist with the builder present (Part 3)
4. **Complete the Guardrails Checklist** — seven questions, all answered in writing, attached to the spec

(Part 4)

5. **Run the "Done" test** — last week's real inputs through the build, four questions answered, failures fixed and retested (Part 5)

Pro Tips (collected)

1. The build path is chosen AFTER Design, not before. Most companies pick their tools first and then try to design a workflow around the tool's limitations. That is backwards.
2. The most common implementation failure is building something that already exists because nobody searched first.
3. Guardrails are what make the workflow trustable. An agent without them gets shut down the first time something goes wrong.

Handoff to Deliver

At the end of Build, you have a working system producing outputs against real inputs, reviewed by the named human supervisor. That is the input Deliver needs. Deliver puts the system into the company's actual operating rhythm — trains the people who use it, changes the handoffs around it, measures the result against the dollar cost Signal put on the constraint, and makes sure the system is owned, not orphaned. A solution that runs is not yet a solution the business has absorbed.

This worksheet is the manual version of the Build Spec Writer instrument and Guardrails Checklist in the Compound Skills Library. The method is the same. The Build Agent on the Bench walks you through it interactively. Both produce the same artifact; the Library gives you the form, the Bench walks you through filling it.